

Yield Curve Factor Models and Housing Market Forecasting

A test of whether Treasury yield curve shape predicts housing starts, mortgage rates, and home prices, using Nelson-Siegel factor extraction and 15 years of out-of-sample forecasting (2011–2025).

Adapted from a research paper for Econ 424: Advanced Analytics, Rutgers University.

Christopher John

TL;DR

them again.

The school

data example of the efficient markets hypothesis: information in the yield curve is already priced in by the time it would be useful for prediction.

Key Findings

Target	Winner	Margin	Statistically Significant?
--------	--------	--------	----------------------------

The home price results

erve Economic Database (FRED): 9 Treasury constant maturity yield series (1-month through 10-year), housing starts (HSTART) and the Case-Shiller national home price index(CSUSHPINSA). The sample spans August 2001 through December 2020. Housing starts and home prices are log-differenced, mortgage rates and yield factors are first-differenced.

Yield Curve Factor Extraction

```
# code cell 3
```

```
# convert maturities into months
maturities = np.array([1, 3, 6, 12, 24, 36, 60, 84, 120])

lam = 0.0009
```

```
S = (1 - np.exp(-lam * tau)) / (lam
C = S - np.exp(-lam * tau)
return np.column_stack([L,S,C])
```

```
# fixed loading matrix
X_ns = ns_loadings(maturities, lam)
```

```
# Fit OLS to extract betas
for data, row in df.iterrows():
    y_curve = row.values
    # OLS: beta = (X'X)^-1 X'y
    beta = np.linalg.lstsq(X_n, y_curve, rcond = None)[0]
    betas.append(beta)

betas_df = pd.DataFrame(betas,
                        index = df.index,
                        columns = ['beta1_L', 'beta2_S', 'beta3_C'])

# add factors to main df
df['f1'] = betas_df['beta1_L']
df['f2'] = betas_df['beta2_S']

#visualize factors
plt.plot(df.index, df['f1'], label='Levels')
plt.plot(df.index, df['f2'], label='Slope')
plt.axhline(0, color='black', linewidth = 0.5, linestyle = '--')
plt.title('Level and Slope curve')
plt.legend()
plt.show()
```

The plot displays two time series over a 20-year period from 2004 to 2024. The 'Levels' series (blue line) shows a general downward trend from around 6 in 2004 to a low of about -2 in 2018, followed by a recovery to around 4 by 2024. The 'Slope' series (orange line) shows a more volatile pattern, starting around -4, peaking near 1 in 2008, dipping to -3 in 2012, and then rising to a peak of about 2 in 2022 before ending near 0. A dashed horizontal line at y=0 serves as a reference.

Five models of increasing complexity, from a naïve

regression output is shown below. The same estimation procedure was applied to all model specifications and dependent variables. Complete model comparisons are summarized in the forecasting evaluation tables presented later. The notebook can be easily modified to display all regression summaries if a more detailed inspection is desired.

```
# code cell 5
#Regressions
for dep_name, (dep_col, dep_lag_col) in dependent_vars.items():
```

```
# Temporarily assign the correct lag to dY_lag for this dependent variable
train_dY["dY_lag"] = train_dY["dep_lag_col"]
train_clean = train_df.copy()

for model_name, predictors in models.items():
    y = train_clean["dep_lag_col"]

    if predictors:
        X = sm.add_constant(train_clean[predictors])
    else:
        X = pd.DataFrame(
            {"const": np.ones(len(train_clean))},
            index=train_clean.index
        )

    model = sm.OLS(y, X).fit()
    results[dep_name][model_name] = model

if dep_name == "HOUST" and model_name == "M2":
    print(f"n(1='60')")
    print(f"dependent variable: {dep_name} | Model: {model_name}")
    print(f"n(1='60')")
    print(model.summary())

=====
Dependent Variable: HOUST | Model: M2
=====
OLS Regression Results
=====
Dep. Variable: dHOUST R-squared: 0.858
Model: OLS Adj. R-squared: 0.841
Method: Least Squares F-statistic: 5.684
Date: Tue, 21 Jul 2026 Prob (F-statistic): 0.0189
Time: 14:29:33 Log-likelihood: 129.36
No. Observations: 110 AIC: -254.3
DF Residuals: 108 BIC: -248.9
DF Model: 1
Covariance Type: nonrobust
=====
coef std err t P>|t| [0.025 0.975]
const -0.017 0.007 -1.611 0.110 -0.026 0.003
```

Tag	-0.2236	0.094	-2.364	0.019	-0.416	-0.056
mnibus:	4.516	Durbin-Watson:	2.042			
rob(Omnibus):	0.105	Jarque-Bera (JB):	3.841			
kew:	-0.362	Prob(JB):	0.147			
urtosis:	2.441	Cond. No.	13.0			

Out-of-Sample Forecasting

code cell 6

```

forecast_results = {}

for dep_name, (dep_col, dep_lag_col) in dependent_vars.items():
    forecast_results[dep_name] = {}

    test_df = df[df.index >= test_start].copy()
    test_df['dy_lag'] = test_df[dep_lag_col]

    for model_name, predictors in models.items():
        forecasts = []
        actuals = []
        dates = []

        for i in range(len(test_df)):
            # expanding window: cutoff grows by one month each iteration,
            # starting right after the training period ends
            cutoff = test_df.index[i - 1] if i > 0 else pd.Timestamp(train_end)
            window = df.loc[:cutoff].copy()

            window['dy_lag'] = window[dep_lag_col]

            y_train = window[dep_col]

            if predictors:
                X_train = sm.add_constant(window[predictors])
            else:
                X_train = pd.DataFrame(
                    {'const': np.ones(len(window))},
                    index=window.index
                )
            # refit on the expanding window, forecast one step ahead
            model_fit = sm.OLS(y_train, X_train).fit()

            # single test point to forecast - pulled out as its own row
            test_point = test_df.iloc[i]

            if predictors:
                X_forecast = pd.DataFrame(
                    [test_point[predictors].values],
                    columns=predictors
                )
            # has_constant='add' since a 1-row frame won't infer the constant on its own
            X_forecast = sm.add_constant(X_forecast, has_constant='add')
            else:
                X_forecast = pd.DataFrame({'const': [1.0]})

            forecast = model_fit.predict(X_forecast)[0]

            forecasts.append(forecast)
            actuals.append(test_df[dep_col].iloc[i])
            dates.append(test_df.index[i])

        forecast_results[dep_name][model_name] = pd.DataFrame({
            'forecast': forecasts,
            'actual': actuals,
            'error': np.array(actuals) - np.array(forecasts)
        }, index=dates)

print(f"Completed forecasts for {dep_name}")

print("All recursive forecasts complete")

```

Completed forecasts for HOUST

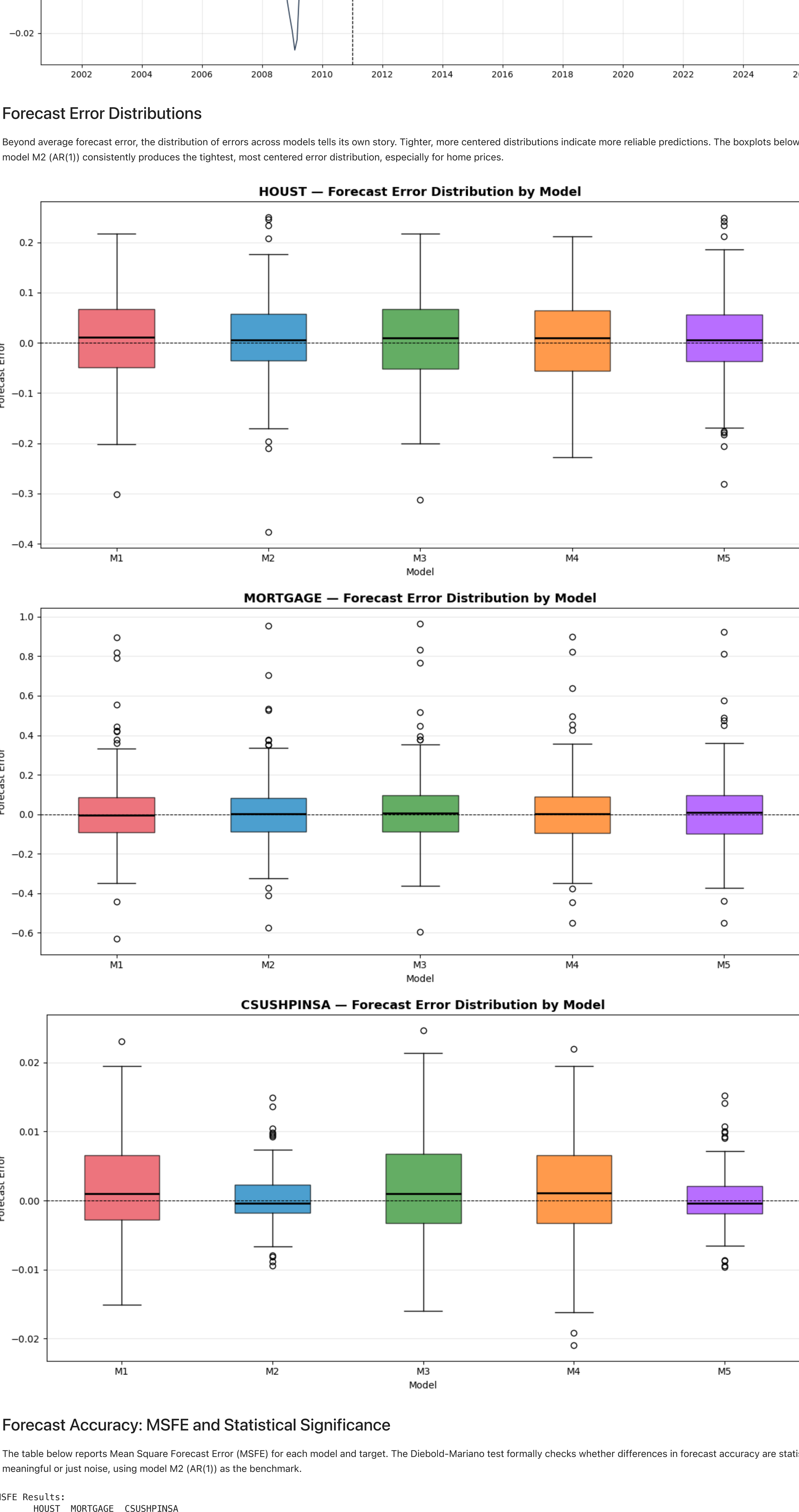
Completed forecasts for MORTGAGE

Completed forecasts for CSUSHPINSA

All recursive forecasts complete

The figure consists of three vertically stacked time series plots, each showing the performance of two forecasting models (M2: AR(1) and M5: Full Model) against actual data. The x-axis for all plots represents time from 2002 to 2016. A vertical dashed line at approximately 2011.5 marks the cutoff point for the training period.

- HOUST — Actual vs M2 and M5:** The y-axis is 'Log Diff Housing Starts' ranging from -0.3 to 0.2. The 'Actual' data (solid black line) shows high volatility. The 'M2: AR(1)' model (dashed blue line) and 'M5: Full Model' (dashed purple line) follow the actual data closely before the cutoff but show significant divergence and bias after the cutoff point.
- MORTGAGE — Actual vs M2 and M5:** The y-axis is 'Change in 30-year Mortgage Rate (pp)' ranging from -0.75 to 0.75. The 'Actual' data (solid black line) shows a clear downward trend followed by a sharp increase. The 'M2: AR(1)' model (dashed blue line) and 'M5: Full Model' (dashed purple line) follow the actual data closely before the cutoff but show significant divergence and bias after the cutoff point.
- CSUSHPINSA — Actual vs M2 and M5:** The y-axis is 'Index' ranging from 0.01 to 0.02. The 'Actual' data (solid black line) shows a clear upward trend. The 'M2: AR(1)' model (dashed blue line) and 'M5: Full Model' (dashed purple line) follow the actual data closely before the cutoff but show significant divergence and bias after the cutoff point.



3	0.008259	0.035976	0.000062
4	0.008300	0.033972	0.000065
5	0.007055	0.034793	0.000015

best model per dependent variable:

IOUST	M2
IORTGAGE	M2
SUSHPINSA	M2

```

code cell 10
# Diebold-Mariano Test
def compute_dm_test(forecast_results, dependent_vars, models, benchmark='M2'):
    dm_results = {}

    for dep_name in dependent_vars.keys():
        dm_results[dep_name] = {}

        actuals = forecast_results[dep_name][benchmark]['actual'].values
        benchmark_forecasts = forecast_results[dep_name][benchmark]['forecast'].values

        for model_name in models.keys():

            if model_name == benchmark:
                dm_results[dep_name][model_name] = {
                    'DM Stat': '-',
                    'p-value': '-',
                    'Conclusion': 'Benchmark'
                }
                continue

            model_forecasts = forecast_results[dep_name][model_name]['forecast'].values

            dm_stat, p_value = dm_test(actuals,
                                      benchmark_forecasts,
                                      model_forecasts,
                                      h=1,
                                      harvey_correction=True,
                                      variance_estimator='bartlett')

            if p_value < 0.05:
                conclusion = f'{model_name} significantly better' if dm_stat > 0 else 'M2 significantly better'
            else:
                conclusion = 'No significant difference'

            dm_results[dep_name][model_name] = {
                'DM Stat': round(dm_stat, 4),
                'p-value': round(p_value, 4),
                'Conclusion': conclusion
            }

    return dm_results

# DM test Results
dm_results = compute_dm_test(forecast_results, dependent_vars, models)
print("\nDiebold-Mariano Test Results:")
for dep_name, comparisons in dm_results.items():
    print(f"({dep_name}):")
    print(pd.DataFrame(comparisons).T)

Diebold-Mariano Test Results:

HOUSE:
DM Stat p-value Conclusion
1 -2.492 0.0136 M2 significantly better
2 - - Benchmark
3 -2.7462 0.0066 M2 significantly better
4 -1.8624 0.0642 No significant difference
5 -0.8205 0.9837 No significant difference

MORTGAGE:
DM Stat p-value Conclusion
1 -1.7015 0.0986 No significant difference
2 - - Benchmark
3 -1.2866 0.2292 No significant difference
4 -0.5551 0.5795 No significant difference
5 -1.2848 0.2299 No significant difference

SUSHPINSA:
DM Stat p-value Conclusion

```

13	-7.0324	0
14	-7.3011	0
15	-1.9965	0.041

Conclusion

This is consistent with the efficient markets hypothesis: publicly available information, including the shape of the yield curve, tends to be priced in quickly, leaving little exploitable